Reinforcement Learning for Tactical Decision-Making in Autonomous Driving

Group Members:

Benwin George, Thilak Sampath Kumaran, Matthew Phillips

Abstract

Unprotected left turns are one of the most difficult everyday driving decisions because the driver must judge gaps in oncoming traffic and commit at the right moment, often under uncertainty and time pressure. In this project, we study this problem as a focused “tactical decision-making” task for an autonomous vehicle, rather than attempting to solve the full self-driving pipeline. We use the highway-env intersection simulation to model an unprotected left-turn scenario and train a reinforcement learning agent to control the ego vehicle’s high-level actions. The agent is trained with Proximal Policy Optimization (PPO) and learns through repeated trial-and-error interactions, receiving rewards for completing the maneuver safely while being penalized for collisions, unsafe proximity to other vehicles, and unnecessary waiting. We evaluate the learned policy using metrics that reflect both safety and efficiency, including success rate, collision rate, timeout rate, and average waiting time, and we compare behavior across reward-function variants to understand key trade-offs. The results demonstrate how reinforcement learning can be applied to a realistic, interaction-heavy driving scenario in simulation and provide a foundation for more advanced tactical decision-making studies in autonomous driving.

Table of Contents

1. Introduction
    - 1.1. Literature Review
        - 1.1.1. Background
        - 1.1.2. Policy gradient Algorithms and PPO
        - 1.1.3. Value-based and other Algorithms
        - 1.1.4. Other Implementations
    - 1.2. Project Scope
    - 1.3. Limitations
2. Methodology
    - 2.1. Overview
    - 2.2. Packages and Frameworks
    - 2.3. Environment
    - 2.4. Model
    - 2.5. Training
    - 2.6. Testing
3. Key Findings
4. Conclusion
5. References

1 Introduction

1.1 Literature Review

1.1.1 Background

Reinforcement Learning (RL) has its theoretical foundations in dynamic programming and optimal control, where early work formalized sequential decisionmaking through concepts such as value functions and Bellman optimality(Bellman, 1957). Classical approaches, including value iteration and policy iteration, established the mathematical framework for RL but relied on complete environment models and were therefore impractical for large or complex real-world problems. The introduction of model-free methods, particularly Q-learning, marked a major shift by enabling agents to learn optimal behavior directly from interaction without explicit knowledge of environment dynamics.

The integration of deep neural networks with RL led to a resurgence of interest in the field in the early 2010s. A key milestone was the introduction of Deep Q-Networks (DQN), which combined Q-learning with deep function approximation to handle high-dimensional state spaces (Mnih et al., 2015). DQN demonstrated that RL could scale to complex perception-based tasks and was initially attractive for control and decision-making problems, including early autonomous driving research. However, DQN is inherently limited to discrete action spaces and exhibits instability and poor scalability in highly stochastic or multi-agent environments, which are characteristic of realistic traffic scenarios.

To overcome these limitations, policy gradient methods were developed to directly optimize parameterized policies rather than relying on value maximization. Early algorithms such as REINFORCE enabled learning in continuous action spaces but suffered from high variance and slow convergence. These issues motivated the development of actor–critic architectures, which combine policy learning with value function estimation to reduce variance and improve training efficiency. Algorithms such as Advantage Actor–Critic (A2C) and Asynchronous Advantage Actor–Critic (A3C) demonstrated improved scalability and supported continuous control, expanding the

applicability of RL to autonomous driving and robotic systems (Mnih et al., 2016). Despite these advances, actor–critic methods remained sensitive to learning rates and the magnitude of policy updates, often leading to unstable training dynamics in complex, interactive environments.

Stability concerns led to a new class of algorithms explicitly designed to constrain policy updates. Trust Region Policy Optimization (TRPO) introduced the idea of limiting the divergence between successive policies using a Kullback–Leibler (KL) constraint, ensuring monotonic improvement during training (Schulman et al., 2015). TRPO significantly improved learning stability and performance in continuous control tasks, but it relied on second-order optimization and complex constraint solving, making it computationally expensive and difficult to implement in practice.

These limitations motivated the development of Proximal Policy Optimization (PPO) in 2017 (Schulman et al., 2017). PPO preserves the stability advantages of trust-region methods while using only first-order optimization through a clipped surrogate objective, greatly simplifying implementation and reducing computational cost. PPO supports both discrete and continuous action spaces and integrates naturally with actor–critic architectures, making it robust in interactive, stochastic environments. As a result, PPO has become one of the most widely adopted RL algorithms in modern applications, including robotics, simulation-based autonomous driving, and large-scale decision-making systems.

In the current era of reinforcement learning, PPO coexists with extensions and alternatives that emphasize sample efficiency, safety, scalability, and multi-agent coordination. Algorithms such as Soft Actor–Critic (SAC), Multi-Agent PPO (MAPPO), and safety-constrained policy optimization methods build upon the same fundamental principles established by PPO. Within this broader landscape, PPO represents a pivotal point in RL's evolution, the convergence of theoretical stability, computational efficiency, and practical usability, making it a strong baseline for studying tactical decisionmaking in complex environments such as autonomous driving.

1.1.2 Gradient Policy Algorithms and PPO

The base algorithm used in our project is the PPO algorithm, which falls under the broader policy gradient paradigm in reinforcement learning. Policy gradient algorithms enable an agent to learn an optimal policy by directly adjusting the parameters of its decision-making process, rather than relying solely on a value-based reward estimation. In essence, the algorithm learns a probability distribution over possible actions given a particular state, allowing the agent to determine which action is most likely to yield the highest expected reward.

The foundation of policy gradient methods is the policy gradient theorem, which expresses the gradient of the expected return $J(\theta)$ with respect to the policy parameters θ as:

$$\nabla_\theta J(\theta)=E_{\pi\theta}[\nabla_\theta \log\pi_\theta(a|s)\, R_t]$$

This formulation, introduced in the REINFORCE algorithm (Sutton et al., 1999), allows the agent to update its policy parameters in the direction that increases the expected cumulative reward. However, REINFORCE suffers from high variance because it relies solely on sampled returns Rt, making learning unstable and slow. To address this, later algorithms introduced baselines and advantage functions to reduce variance. The advantage function $A_t = Q_t - V_t$ measures how much better an action is compared to the average, leading to the Actor–Critic framework, where the actor updates the policy and the critic estimates the value function.

Despite these improvements, early policy gradient methods still faced instability due to large, unconstrained updates. This motivated the development of Trust Region Policy Optimization (TRPO) (Schulman et al., 2015), which introduced a constraint on the step size of policy updates using the Kullback–Leibler (KL) divergence:

$$\max_\theta E[(\pi_\theta(a|s)/\pi_{\theta old}(a|s))A_t] \text{ s.t. } D_{KL}(\pi_{\theta old} \,//\, \pi_\theta) \leq \delta$$

This ensures that the new policy does not deviate too far from the old one, improving stability and preventing catastrophic performance drops. However, TRPO's constrained optimization is computationally expensive and complex to implement.

To simplify this, PPO (Schulman et al., 2017) replaced the hard constraint with a clipped surrogate objective, which penalizes updates that move the policy too far from the previous iteration:

$$L^{CLIP}(\theta)=E_t[\min(r_t(\theta)A_t,\mathrm{clip}(r_t(\theta),1-\epsilon,1+\epsilon)A_t)]$$

where $r_t(\theta)=\pi_\theta(a_t|s_t)/\pi_{\theta old}(a_t|s_t)$ is the probability ratio between the new and old policies. The clipping term ensures that updates remain "proximal," preventing overly aggressive changes while maintaining learning efficiency.

Each evolution in the policy gradient family addressed a specific limitation of its predecessor:

- REINFORCE introduced the basic gradient estimation but suffered from high variance.
- Actor–Critic reduced variance by incorporating a learned value baseline.
- TRPO improved stability through trust-region constraints but was computationally heavy.
- PPO achieved similar stability with a simpler, more efficient clipping mechanism.

PPO's balance of stability, simplicity, and performance makes it the preferred choice for complex, continuous control tasks. It retains the theoretical grounding of policy gradients while offering practical advantages in training speed and robustness, which is why it serves as the foundation for this project.

1.1.3 Value based and other Algorithms

While policy gradient algorithms such as PPO focus on directly optimizing the policy, value-based methods take a different approach by estimating the expected return of

each action and deriving the optimal policy from those estimates. The most well-known example is Deep Q-Learning (DQN), which approximates the action-value function Q(s,a) using a neural network and updates it according to the Bellman equation:

$$Q(s,a)=r+\gamma \max_{a'} Q(s',a')$$

This formulation allows the agent to learn which actions yield the highest long-term reward, but it relies on discrete action spaces and can become unstable when the Q-values are overestimated. To mitigate this, variants such as Double DQN (Van Hasselt et al., 2016) and Dueling DQN (Wang et al., 2016) were introduced, improving stability and sample efficiency.

In contrast, policy gradient methods are better suited for continuous control tasks, where the action space cannot be easily discretized. PPO's clipped objective ensures smoother updates and prevents the oscillations that often occur in value-based learning. Moreover, PPO integrates both policy and value learning through its actor–critic structure, combining the strengths of both paradigms.

Another modern hybrid approach is Soft Actor–Critic (SAC) (Haarnoja et al.,2018), which extends the actor–critic framework by introducing an entropy term to encourage exploration. SAC optimizes both the expected reward and the entropy of the policy:

$$J(\pi)=\sum_t E_{(st,at)\sim\pi}[r(s_t,a_t) + \alpha H(\pi(\cdot \mid s_t))]$$

This formulation ensures that the agent remains exploratory even after convergence, often outperforming PPO in highly stochastic environments.

In summary, value-based algorithms excel in discrete, well-defined environments, while policy gradient algorithms, especially PPO, dominate in continuous and complex domains. PPO's balance of stability, efficiency, and adaptability makes it a practical choice for real-world applications such as autonomous driving, robotics, and dynamic traffic modeling, aligning perfectly with the goals of this project.

1.1.4 Other Implementations

Navigating unsignalized intersections represents a significant challenge for autonomous vehicles due to the high-conflict nature of traffic and the need to balance safety with efficiency. Traditional rule-based approaches, such as Time-to-Collision (TTC), often struggle in dense traffic because they assume constant speeds for all participants and do not account for the complex intentions of other drivers. In such scenarios, conservative rule-based systems frequently result in "undefined waits" or unnecessary delays. Recent research has turned to Deep Reinforcement Learning (DRL) to develop more "behavior-aware" agents capable of negotiation.

Capasso et al. (2021) addressed this by training agents in a multi-agent fashion, allowing road rules to emerge naturally through interaction. Their system uses a Delayed-A3C (D-A3C) algorithm, which improves upon standard A3C by gathering updates throughout an entire episode before updating the global network, leading to better performance in continuous control tasks like acceleration and steering. The state representation is particularly innovative, utilizing four grayscale semantic images (navigable space, path, obstacles, and traffic signs). Crucially, they embedded priority knowledge directly into the obstacle channel: vehicles with a higher right-of-way were rendered in black, while those with lower priority were white. This allowed the agent to learn the "priority to the right" rule without explicit programming. The module achieved success rates exceeding 99% in synthetic environments and demonstrated remarkable generalization when tested on real-world traffic data from the inD dataset, proving it could handle human driving behaviors not seen during training.

While the first study focused on rule emergence, Liu et al. (2022) focused on the trade-off between safety and efficiency through a multi-task framework. They employed the Twin Delayed Deep Deterministic policy gradient (TD3) algorithm to navigate scenarios involving turning left, right, or going straight. To ensure safety during the RL exploration phase, where random actions can be catastrophic, the authors introduced

an analytical safety layer. This layer uses a neural network trained on offline data to predict safety constraints (inspired by TTC) and analytically modifies dangerous actions before they are executed in the environment. Furthermore, they integrated a social attention module to capture dependencies between the ego-vehicle and the most relevant nearby "social" vehicles. This module improved performance and provided interpretability, as attention weights could be visualized to show exactly which vehicle influenced the agent’s decision. Their results in SUMO and CARLA simulators showed significant gains over previous multi-task models, including a 24% increase in traffic efficiency for complex left turns.

These studies provide a technical roadmap for implementing an autonomous navigation agent using a base PPO algorithm in environments such as highway-env. Capasso et al. demonstrate that representing traffic rules within the state space, such as through multi-channel semantic data, can help an agent learn complex priorities. Meanwhile, Liu et al. illustrate the benefit of using vectorized rewards to separate feedback for specific maneuvers (e.g., turning left vs. straight) and common goals like traffic efficiency. For a project focused on reward shaping, these papers suggest that focusing on target speed tracking, path alignment through localization rewards, and heavily penalizing collisions based on priority can help a PPO-based agent navigate complex intersections safely without becoming overly conservative.

1.2 Project Scope

Implement an RL model that can perform a left turn consistently in an environment that has no signaling for drivers. The model will have to consider other drivers' speed and positions to decide what to do next. The model will be able to perform the following actions in the environment, wait, move forward, and initiate the turn. The model must be safe (avoid collisions), efficient (does not wait too long), and robust (can handle variance in the environment). If time permits, other traffic challenges, like highway merging, will be performed and tested to see if the model can handle a robust problem set.

1.3 Limitations

The primary limitation of this project is that all experiments are conducted entirely within a simulated environment. While simulation provides a safe and controllable setting for training and evaluation, it inherently constrains variability and realism, as the simulator resolves interactions in a consistent and predefined manner across episodes. This reduces exposure to unpredictable real-world factors such as sensor noise, hardware failures, or unexpected driver behavior. In addition, the simulation environment offers a limited set of driving scenarios and does not support extended, continuous driving comparable to real-world conditions. The behavior of surrounding vehicles is governed by simplified models provided by the simulator, which may not fully capture the diversity, irregularities, and edge cases present in human driving behavior. Consequently, the generalizability of the learned policy beyond the simulated domain is not guaranteed.

2 Methodology

2.1 Overview

This project treats a difficult driving situation—an unprotected left turn at an intersection—as a focused decision-making problem rather than trying to solve “full self-driving.” In the simulation, the ego vehicle repeatedly encounters oncoming traffic and must decide when to wait, when to creep forward, and when to commit to the turn. We train a reinforcement learning (RL) agent using Proximal Policy Optimization (PPO) so that, through trial-and-error experience, it learns a policy that balances safety (avoiding collisions and risky gaps) with efficiency (not waiting unnecessarily long). After training, we evaluate the learned policy over many episodes and summarize performance using metrics like success rate, collision rate, timeout rate, and average waiting time.

2.2 Packages and Frameworks

These are the packages and frameworks that were used in the project:

- Python : the main implementation language for training, evaluation, and utilities.
- Gymnasium: provides the standard RL environment interface (reset/step APIs, spaces, etc.).
- highway-env: supplies the traffic simulation environment, specifically an intersection scenario suitable for tactical decision-making research.
- Stable-Baselines3: provides the PPO algorithm implementation used for training.
- NumPy: used for numeric computations (e.g., distance calculations and aggregation of episode statistics).
- Pygame : used for visualizing rollouts of a trained policy with a simple HUD-style display.

2.3 Environment

We use the highway-env intersection task (intersection-v0) as the main experimental environment. The environment simulates an intersection with multiple vehicles and realistic interactions, and it exposes state information that allows the agent to reason about the local traffic situation.

During training, the environment reports:

Observation space: a continuous tensor with shape (15, 7) (i.e., a structured observation rather than a single 1D feature vector).

Action space: Discrete(3), meaning the agent selects among three high-level actions at each decision step.

An episode ends when the ego vehicle either completes the maneuver successfully, collides, or times out (e.g., the scenario ends without reaching a successful terminal condition). For evaluation, we also track “waiting” behavior using a speed threshold

(steps where the vehicle speed is near zero are counted as waiting), allowing us to quantify how cautious vs. aggressive the learned policy is.

2.4 Model

We train a PPO agent using Stable-Baselines3 with an MLP-based policy (MlpPolicy). PPO is a widely used policy-gradient method that is generally stable for environments like this where the agent must learn from repeated interaction and delayed outcomes (e.g., a decision to go might only reveal its quality moments later through success or a collision).

Training configuration (core settings used in code):

Algorithm: PPO (Stable-Baselines3)

Policy: MlpPolicy

Rollout length (n_steps): 2048 steps per iteration

Batch size: 64

Epochs per update: 10

Discount factor (gamma): 0.99

GAE (gae_lambda): 0.95

Clip range: 0.2

Entropy coefficient: 0.01

Learning rate: 3e-4

Random seed: 42 (to make experiments more reproducible)

Reward design is treated as an experimental lever. In addition to a baseline reward, the project includes reward variants to test safety/efficiency tradeoffs:

baseline: default environment reward.

collision_strong: increases the penalty for collisions to emphasize safety.

near_miss_penalty: adds an extra penalty when the ego vehicle gets too close to other vehicles, encouraging larger safety margins.

waiting_penalty: adds a small penalty when the ego vehicle is essentially stopped, encouraging efficiency and reducing “over-cautious” policies.

2.5 Training

We trained a Proximal Policy Optimization (PPO) agent in the highway-env intersection-v0 simulator for unprotected left-turn decision making.

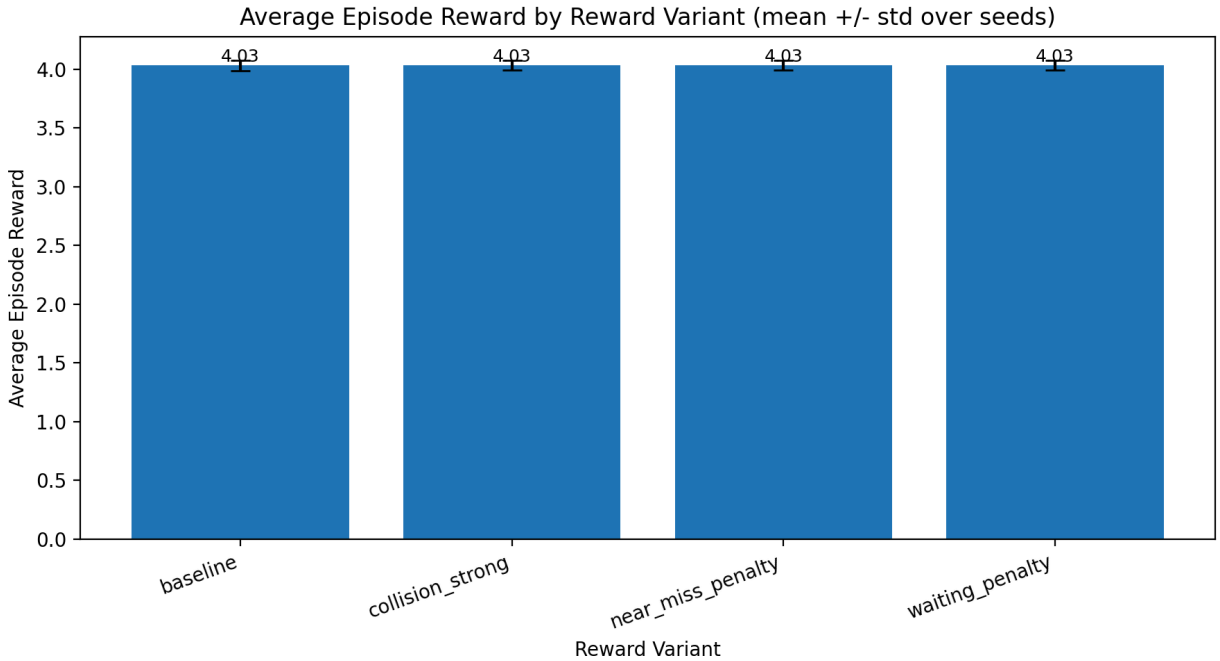
To study reward design effects, we trained four models:

- baseline
- collision_strong
- near_miss_penalty
- waiting_penalty

Each model was trained with the same PPO setup and timesteps to ensure a fair comparison.

The reward-variant experiments were designed to test whether stronger safety-oriented shaping can reduce collision outcomes without heavily sacrificing task completion.

In the final reward-variant experiment, each model (baseline, collision_strong, near_miss_penalty, waiting_penalty) was trained for 100,000 timesteps using the same PPO hyperparameters to ensure a fair comparison across reward designs.



2.6 Testing

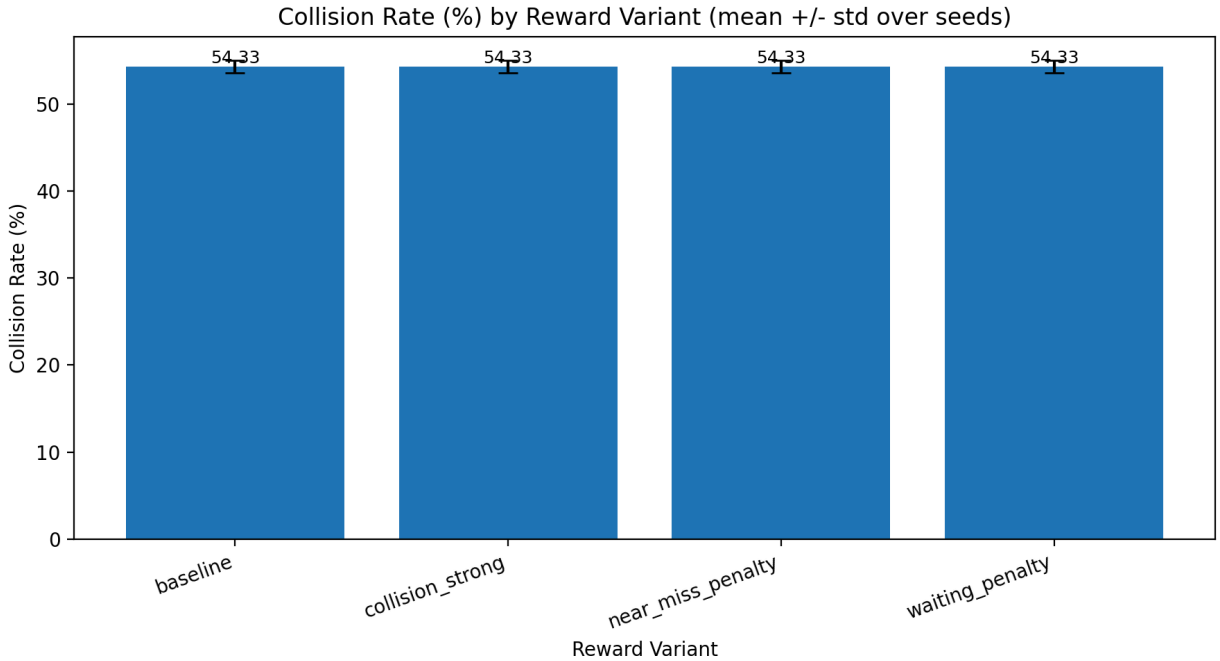
Testing was performed using the same environment and a standardized multi-seed evaluation protocol.

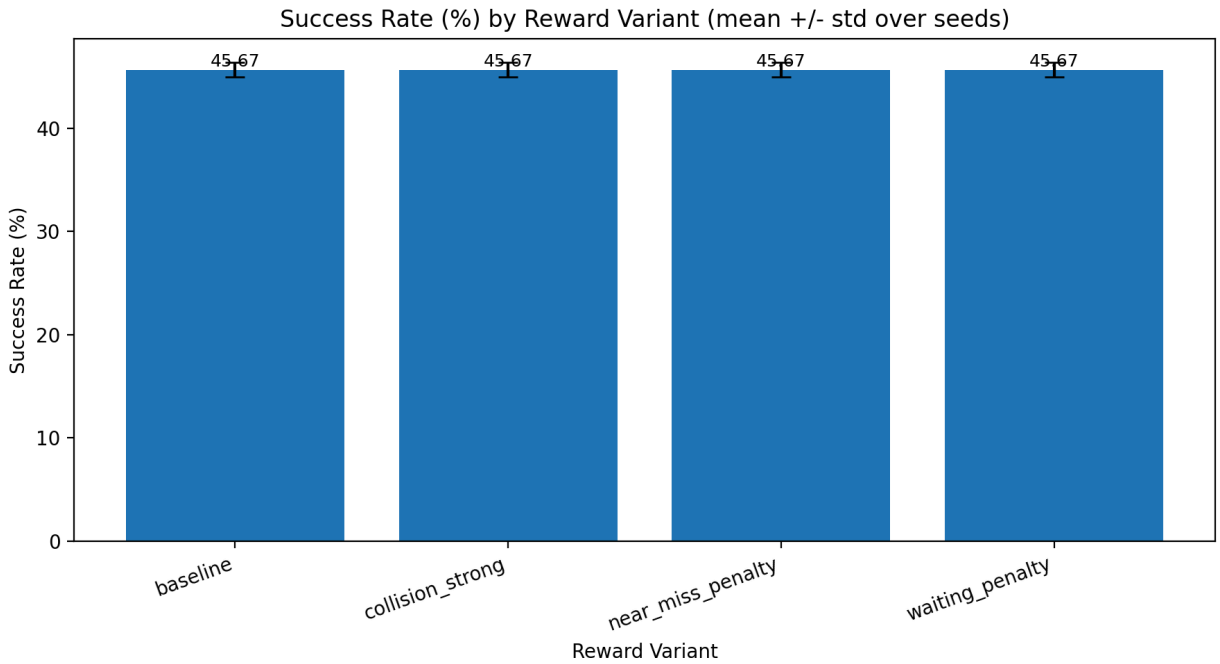
We evaluated each trained model using a fixed multi-seed protocol with seed set {42, 142, 242} and 500 episodes per seed (1,500 episodes per model total).

We recorded:

- Collision rate (%)
- Success rate (%)
- Average waiting time (s)
- Average episode reward

Results were aggregated by reward variant using mean (and standard deviation) across seeds.





3 Key Findings

1. All four reward variants produced very similar results under the current setup.
2. Mean collision rate was approximately 54.33% for all variants.

3. Mean success rate was approximately 45.67% for all variants.
4. Average reward was also nearly identical across variants.
5. Average waiting time was effectively 0.00 s across variants using the current waiting threshold, so this metric did not distinguish policies.

Variant	Collision Mean (%)	Success Mean (%)	Waiting Mean (s)	Reward Mean
waiting_penalty	54.333333	45.666667	0.000000	4.033316
baseline	54.333333	45.666667	0.000000	4.031983
near_miss_penalty	54.333333	45.666667	0.000000	4.033060
collision_strong	54.333333	45.666667	0.000000	4.033316

Across the tested setup, all four reward variants produced nearly identical safety and task-completion outcomes, indicating limited sensitivity to the current reward-shaping magnitudes.

4 Conclusion

This project demonstrated an end-to-end PPO pipeline for tactical autonomous-driving decisions in an unprotected left-turn scenario.

The agent can be trained and evaluated systematically, and we successfully implemented reward-variant experiments with multi-seed testing.

However, under current settings, the tested reward variants did not yield significant differences in collision or success outcomes.

This indicates the need for stronger or better-targeted reward shaping to improve safety performance.

Future work should focus on:

- Stronger safety penalties (collision and near-miss),
- More sensitive efficiency metrics (revising waiting-time definition),
- Longer training budgets and additional seeds,
- Potential migration to newer environment versions for robustness.

5 References

- Bellman, R. (1957). Dynamic programming. Princeton University Press.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). *Human-level control through deep reinforcement learning*. Nature, 518(7540), 529–533. https://doi.org/10.1038/nature14236
- Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., & Kavukcuoglu, K. (2016). *Asynchronous methods for deep reinforcement learning*. Proceedings of the 33rd International Conference on Machine Learning (ICML), 1928–1937. https://proceedings.mlr.press/v48/mniha16.pdf
- Schulman, J., Levine, S., Abbeel, P., Jordan, M., & Moritz, P. (2015). *Trust region policy optimization*. Proceedings of the 32nd International Conference on Machine Learning (ICML), 1889–1897. https://proceedings.mlr.press/v37/schulman15.pdf
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). *Proximal policy optimization algorithms*. arXiv. https://arxiv.org/abs/1707.06347
- Sutton, R. S., McAllester, D., Singh, S., & Mansour, Y. (1999). *Policy Gradient Methods for Reinforcement Learning with Function Approximation*. Advances in Neural Information Processing Systems.
- Konda, V. R., & Tsitsiklis, J. N. (2000). *Actor-Critic Algorithms*. Advances in Neural Information Processing Systems.
- Van Hasselt, H., Guez, A., & Silver, D. (2016). *Deep Reinforcement Learning with Double Q-Learning*. Proceedings of the 30th AAAI Conference on Artificial Intelligence.
- Wang, Z., Schaul, T., Hessel, M., Hasselt, H. V., Lanctot, M., & Silver, D. (2016). *Dueling Network Architectures for Deep Reinforcement Learning*. Proceedings of the 33rd International Conference on Machine Learning (ICML).

- Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*. Proceedings of the 35th International Conference on Machine Learning (ICML).
- Capasso, A. P., Maramotti, P., Dell'Eva, A., & Broggi, A. (2021). *End-to-end intersection handling using multi-agent deep reinforcement learning*. *arXiv*. https://arxiv.org/abs/2104.13617
- Liu, Y., Gao, Y., Zhang, Q., Ding, D., & Zhao, D. (2022). *Multi-task safe reinforcement learning for navigating intersections in dense traffic. arXiv*. https://arxiv.org/abs/2202.09644